

Rancangan Database — Sistem Reservasi & Pelaporan Fasilitas Kampus (PPK 2026)

Dokumen ini berisi rancangan skema database beserta asumsi perancangan untuk platform reservasi dan pelaporan kerusakan fasilitas kampus, disusun berdasarkan deskripsi project, Information Architecture (IA), dan userflow (login/autentikasi, pelaporan kerusakan, reservasi).

Daftar Isi

- Struktur Tabel
- Relasi Antar Tabel
- Entity Relationship Overview
- Asumsi Perancangan
- Catatan Implementasi Tambahan

Struktur Tabel

Setiap kolom dilengkapi tipe data, opsionalitas (NULL/NOT NULL), dan default value.

1. users

Menyimpan data seluruh aktor sistem (pengguna, petugas, admin).

Kolom	Tipe	Null?	Default	Keterangan
id	PK	NOT NULL	auto	
name	VARCHAR	NOT NULL	-	
email	VARCHAR	NOT NULL	-	UNIQUE
password	VARCHAR	NOT NULL	-	Format hash
role	ENUM('pengguna','petugas','admin')	NOT NULL	-	
user_type	ENUM('mahasiswa','dosen','staf')	NULL	NULL	Hanya diisi jika role = 'pengguna'
status	ENUM('pending','verified','rejected')	NOT NULL	'pending' (registrasi mandiri) / 'verified' (dibuat admin)	Status verifikasi akun
account_status	ENUM('aktif','nonaktif')	NULL	NULL	Status aktif/nonaktif akun, terpisah dari status . Lihat CHECK constraint di bawah.
deleted_at	DATETIME	NULL	NULL	Kolom soft delete (Laravel SoftDeletes). Diisi otomatis saat akun "dihapus" — baris tetap ada di DB agar riwayat

Kolom	Tipe	Null?	Default	Keterangan
				reservasi/laporan/log tidak rusak.
<code>created_at</code>	DATETIME	NOT NULL	<code>NOW()</code>	
<code>updated_at</code>	DATETIME	NOT NULL	<code>NOW()</code>	

CHECK constraint: `account_status` hanya boleh terisi ('aktif'/'nonaktif') ketika `status` = 'verified'; selain itu wajib NULL.

```

CHECK (
  (status = 'verified' AND account_status IN ('aktif','nonaktif'))
  OR
  (status <> 'verified' AND account_status IS NULL)
)

```

Saat `status` berubah menjadi 'verified' (admin meng-ACC), `account_status` otomatis diisi default 'aktif'. Admin bisa toggle ke 'nonaktif' kapan saja setelah itu (mirip fitur nonaktifkan pada `facilities`), tapi tidak bisa dilakukan pada akun yang masih 'pending'/'rejected'.

2. locations

Data master hierarki lokasi (Universitas / Fakultas → Prodi → Gedung → Ruangan).

Kolom	Tipe	Null?	Default	Keterangan
<code>id</code>	PK	NOT NULL	auto	
<code>scope_level</code>	ENUM('universitas','fakultas')	NOT NULL	'fakultas'	Penanda apakah lokasi ini tingkat universitas (fasilitas umum kampus) atau tingkat fakultas
<code>fakultas</code>	VARCHAR	NULL	NULL	Wajib diisi jika <code>scope_level</code> = 'fakultas'; wajib NULL jika <code>scope_level</code> = 'universitas'
<code>prodi</code>	VARCHAR	NULL	NULL	
<code>gedung</code>	VARCHAR	NULL	NULL	Kosong untuk fasilitas outdoor
<code>ruangan</code>	VARCHAR	NULL	NULL	Kosong untuk fasilitas outdoor

CHECK constraint: `fakultas` hanya boleh kosong jika lokasi memang didaftarkan sebagai tingkat universitas. Karena `scope_level` sudah menjadi penanda eksplisit, NULL di sini tidak ambigu — jelas berarti "sengaja tidak ada fakultas", bukan "data belum diisi".

```

CHECK (
  (scope_level = 'universitas' AND fakultas IS NULL)
  OR
  (scope_level = 'fakultas' AND fakultas IS NOT NULL)
)

```

Di form admin, `scope_level` dipakai sebagai toggle pilihan "Universitas / Fakultas" — saat admin memilih "Universitas", field `fakultas` otomatis disembunyikan dan dikirim sebagai `NULL`.

3. `facility_types`

Data master kategori/jenis fasilitas.

Kolom	Tipe	Null?	Default	Keterangan
<code>id</code>	PK	NOT NULL	auto	
<code>name</code>	VARCHAR	NOT NULL	-	mis. Ruang Kelas, Laboratorium, Lapangan

4. `facilities`

Data utama fasilitas kampus.

Kolom	Tipe	Null?	Default	Keterangan
<code>id</code>	PK	NOT NULL	auto	
<code>name</code>	VARCHAR	NOT NULL	-	
<code>type_id</code>	FK → <code>facility_types.id</code>	NOT NULL	-	
<code>location_id</code>	FK → <code>locations.id</code>	NOT NULL	-	
<code>capacity</code>	INT	NOT NULL	-	
<code>description</code>	TEXT	NULL	NULL	
<code>status</code>	ENUM('aktif','dalam perbaikan','nonaktif')	NOT NULL	'aktif'	
<code>deleted_at</code>	DATETIME	NULL	NULL	Kolom soft delete (Laravel <code>SoftDeletes</code>). Diisi otomatis saat fasilitas "dihapus" — baris tetap ada di DB agar riwayat reservasi/laporan/foto tidak rusak.
<code>created_at</code>	DATETIME	NOT NULL	<code>NOW()</code>	
<code>updated_at</code>	DATETIME	NOT NULL	<code>NOW()</code>	

5. `facility_photos`

Dokumentasi multi-foto fasilitas.

Kolom	Tipe	Null?	Default	Keterangan
<code>id</code>	PK	NOT NULL	auto	
<code>facility_id</code>	FK → <code>facilities.id</code>	NOT NULL	-	
<code>file_path</code>	VARCHAR	NOT NULL	-	
<code>is_primary</code>	BOOLEAN	NOT NULL	<code>FALSE</code>	Penanda foto utama (thumbnail)
<code>uploaded_at</code>	DATETIME	NOT NULL	<code>NOW()</code>	

6. reservations

Data transaksi reservasi fasilitas.

Kolom	Tipe	Null?	Default	Keterangan
<code>id</code>	PK	NOT NULL	auto	
<code>user_id</code>	FK → <code>users.id</code>	NOT NULL	-	Pemesan
<code>facility_id</code>	FK → <code>facilities.id</code>	NOT NULL	-	Fasilitas yang dipesan
<code>reservation_date</code>	DATE	NOT NULL	-	
<code>start_time</code>	TIME	NOT NULL	-	Validasi 07.00–20.00, kelipatan 30 menit di server
<code>end_time</code>	TIME	NOT NULL	-	Validasi sama seperti <code>start_time</code>
<code>purpose</code>	TEXT	NOT NULL	-	
<code>status</code>	ENUM('pending', 'approved', 'rejected', 'cancelled')	NOT NULL	'pending'	
<code>cancellation_reason</code>	TEXT	NULL	NULL	Diisi saat status → 'cancelled' oleh petugas
<code>created_at</code>	DATETIME	NOT NULL	NOW()	
<code>updated_at</code>	DATETIME	NOT NULL	NOW()	

Aturan availability: slot dianggap "occupied" (tidak bisa dipesan ulang) selama reservasi berstatus `pending` maupun `approved`. Ini mencegah dua pengguna mengajukan slot yang sama secara bersamaan sebelum petugas sempat memproses. Lihat [Asumsi Perancangan](#) poin 10.

7. reports

Data laporan kerusakan fasilitas.

Kolom	Tipe	Null?	Default	Keterangan
<code>id</code>	PK	NOT NULL	auto	
<code>user_id</code>	FK → <code>users.id</code>	NOT NULL	-	Pelapor

Kolom	Tipe	Null?	Default	Keterangan
<code>facility_id</code>	FK → <code>facilities.id</code>	NOT NULL	-	Fasilitas bermasalah
<code>category</code>	VARCHAR	NOT NULL	-	
<code>description</code>	TEXT	NULL	NULL	Boleh kosong selama status <code>'draft'</code>
<code>status</code>	ENUM(<code>'draft'</code> , <code>'baru'</code> , <code>'diproses'</code> , <code>'selesai'</code> , <code>'ditolak'</code>)	NOT NULL	<code>'draft'</code>	
<code>resolution_note</code>	TEXT	NULL	NULL	Diisi saat laporan ditutup
<code>resolved_at</code>	DATETIME	NULL	NULL	Diisi saat status → <code>'selesai'</code> / <code>'ditolak'</code>
<code>created_at</code>	DATETIME	NOT NULL	<code>NOW()</code>	
<code>updated_at</code>	DATETIME	NOT NULL	<code>NOW()</code>	

8. `report_photos`

Dokumentasi bukti kerusakan.

Kolom	Tipe	Null?	Default	Keterangan
<code>id</code>	PK	NOT NULL	auto	
<code>report_id</code>	FK → <code>reports.id</code>	NOT NULL	-	
<code>file_path</code>	VARCHAR	NOT NULL	-	
<code>uploaded_at</code>	DATETIME	NOT NULL	<code>NOW()</code>	

9. `status_logs`

Audit trail / rekam jejak perubahan status.

Kolom	Tipe	Null?	Default	Keterangan
<code>id</code>	PK	NOT NULL	auto	
<code>reservation_id</code>	FK → <code>reservations.id</code>	NULL	NULL	Tepat 1 dari 3 FK ini yang terisi per baris
<code>report_id</code>	FK → <code>reports.id</code>	NULL	NULL	Tepat 1 dari 3 FK ini yang terisi per baris
<code>facility_id</code>	FK → <code>facilities.id</code>	NULL	NULL	Tepat 1 dari 3 FK ini yang terisi per baris
<code>changed_by_user_id</code>	FK → <code>users.id</code>	NOT NULL	-	Petugas/Admin yang melakukan perubahan
<code>old_status</code>	VARCHAR	NOT NULL	-	
<code>new_status</code>	VARCHAR	NOT NULL	-	
<code>note</code>	TEXT	NULL	NULL	
<code>created_at</code>	DATETIME	NOT NULL	<code>NOW()</code>	

Kolom nullable di seluruh skema (rekap cepat): `users.user_type` · `users.account_status` (nullable, tapi *wajib* terisi begitu `status = 'verified'` — lihat CHECK constraint) · `users.deleted_at` (soft delete) · `locations.fakultas` (nullable jika

scope_level = 'universitas') · locations.prodi/gedung/ruangan · facilities.description · facilities.deleted_at (soft delete) · reservations.cancellation_reason · reports.description/resolution_note/resolved_at · status_logs.reservation_id/report_id/facility_id/note

Relasi Antar Tabel

Notasi kardinalitas & opsionalitas

- **1** = kaki "satu" (sisi yang punya Primary Key, wajib unik).
- **M** = kaki "banyak" (sisi yang menyimpan Foreign Key, boleh berulang). Kaki "M" selalu berada di tabel yang punya kolom FK.
- **Garis lurus (tegas/solid)** = partisipasi **wajib (mandatory)**. Artinya baris di sisi tersebut **tidak boleh ada tanpa** pasangannya — di level skema ini terlihat dari kolom FK yang **NOT NULL**. Contoh: setiap **reservations** **wajib** merujuk ke **facilities** (**facility_id NOT NULL**) — tidak mungkin ada reservasi tanpa fasilitas.
- **Garis putus-putus (dashed)** = partisipasi **opsional**. Artinya baris di sisi tersebut **boleh ada tanpa** pasangannya — terlihat dari kolom FK yang **nullable**. Contoh: baris **status_logs** boleh saja tidak terkait ke **reservations** (**reservation_id NULL**) karena log itu bisa jadi milik **reports** atau **facilities** sebagai gantinya.
- Pada sisi "1" (induk), partisipasi biasanya juga wajib secara logis (setiap **facilities** bisa saja tidak punya reservasi sama sekali → opsional dari sudut pandang induk), tapi yang paling relevan untuk implementasi FK adalah opsionalitas di sisi "M" (anak), karena itu yang menentukan NULL/NOT NULL di kolom FK.

Tabel Induk (1)	Tabel Anak (M)	Kardinalitas	FK berada di	Tipe Garis	Penjelasan
users	reservations	1 : M	reservations.user_id (NOT NULL)	Lurus (wajib)	Satu user bisa punya banyak reservasi; setiap reservasi wajib punya satu pemesan.
facilities	reservations	1 : M	reservations.facility_id (NOT NULL)	Lurus (wajib)	Satu fasilitas bisa dipesan berkali-kali di waktu yang berbeda (mis. 07.00–08.00 oleh X, 09.00–10.00 oleh Y); setiap reservasi wajib merujuk satu fasilitas. Tetap 1:M, bukan 1:1 — lihat catatan di bawah tabel.
users	reports	1 : M	reports.user_id (NOT NULL)	Lurus (wajib)	Satu user bisa membuat banyak laporan; setiap laporan wajib punya satu pelapor.

Tabel Induk (1)	Tabel Anak (M)	Kardinalitas	FK berada di	Tipe Garis	Penjelasan
<code>facilities</code>	<code>reports</code>	1 : M	<code>reports.facility_id</code> (NOT NULL)	Lurus (wajib)	Satu fasilitas bisa dilaporkan berkali-kali; setiap laporan wajib merujuk satu fasilitas.
<code>facility_types</code>	<code>facilities</code>	1 : M	<code>facilities.type_id</code> (NOT NULL)	Lurus (wajib)	Satu jenis fasilitas dipakai banyak fasilitas; setiap fasilitas wajib punya satu jenis.
<code>locations</code>	<code>facilities</code>	1 : M	<code>facilities.location_id</code> (NOT NULL)	Lurus— Lurus (wajib di kedua sisi)	Setiap fasilitas wajib ada di satu lokasi (FK NOT NULL), dan setiap lokasi yang terdaftar wajib punya minimal satu fasilitas (aturan bisnis tim: lokasi tidak boleh "kosong"). Lihat catatan penegakan mandatory-mandatory di bawah tabel.
<code>facilities</code>	<code>facility_photos</code>	1 : M	<code>facility_photos.facility_id</code> (NOT NULL)	Lurus (wajib)	Satu fasilitas bisa punya banyak foto; setiap foto wajib milik satu fasilitas.
<code>reports</code>	<code>report_photos</code>	1 : M	<code>report_photos.report_id</code> (NOT NULL)	Lurus— Lurus (wajib di kedua sisi)	Setiap foto wajib milik satu laporan (FK NOT NULL), dan setiap laporan yang dibuat wajib punya minimal satu foto bukti (aturan bisnis tim: laporan tanpa foto

Tabel Induk (1)	Tabel Anak (M)	Kardinalitas	FK berada di	Tipe Garis	Penjelasan
					tidak boleh disubmit). Lihat catatan penegakan mandatory-mandatory di bawah tabel.
reservations	status_logs	1 : M	status_logs.reservation_id (NULL)	Putus-putus (opsional)	Satu reservasi bisa punya banyak baris log; sebaliknya, satu baris status_logs tidak wajib terkait reservasi (bisa jadi terkait reports atau facilities).
reports	status_logs	1 : M	status_logs.report_id (NULL)	Putus-putus (opsional)	Satu laporan bisa punya banyak baris log; satu baris status_logs tidak wajib terkait laporan.
facilities	status_logs	1 : M	status_logs.facility_id (NULL)	Putus-putus (opsional)	Satu fasilitas bisa punya banyak baris log (mis. aktif ↔ dalam perbaikan); satu baris status_logs tidak wajib terkait fasilitas.
users	status_logs	1 : M	status_logs.changed_by_user_id (NOT NULL)	Lurus (wajib)	Satu petugas/admin bisa membuat banyak entri log; setiap entri log wajib dibuat oleh satu user (tidak ada log anonim).

Fasilitas–Reservasi tetap 1:M, bukan 1:1

Relasi mandatory-mandatory (`locations`↔`facilities` dan `reports`↔`report_photos`)

Karena itu, sisi "induk wajib punya anak" pada kedua relasi ini **tidak ditegakkan oleh skema/FK**, melainkan oleh aplikasi:

- Lihat juga Catatan Implementasi Tambahan poin 4.

Tidak ada relasi Many-to-Many (M:M) di skema ini — semua relasi bersifat 1:M langsung tanpa tabel pivot.

Entity Relationship Overview

```

users —1—M— reservations —M—1— facilities —1—M— facility_photos
|                                     ||      |
|                                     ||      M—1— facility_types
|                                     ||      L=M=1 locations (mandatory-mandatory)
|
|——1—M— reports ==M==1 report_photos (mandatory-mandatory)
    |
    └── (facilities, lihat atas)

status_logs (M) —T—M---1— reservations (opsional - FK reservation_id nullable)
                  ├──M---1— reports     (opsional - FK report_id nullable)
                  ├──M---1— facilities   (opsional - FK facility_id nullable)
                  └──M—1— users (changed_by_user_id, WAJIB - FK NOT NULL)

```

9 / 12

(*locations*↔*facilities* dan *reports*↔*report_photos*) karena wajib di kedua sisi — lihat penjelasan lengkap di bagian tabel relasi.

Asumsi Perancangan

- 1. Autentikasi & Tipe Akun** Akun Admin dan Petugas langsung didaftarkan oleh Admin sehingga statusnya otomatis *'verified'*. Akun Pengguna (mahasiswa, dosen, staf) hasil registrasi mandiri akan berstatus *'pending'* hingga diverifikasi Admin. Kolom *user_type* dibuat nullable karena klasifikasi tersebut hanya relevan untuk role *'pengguna'*.
- 2. Status Verifikasi vs Status Aktif/Nonaktif Akun** *status* (verifikasi) dan *account_status* (aktif/nonaktif) dipisah menjadi dua kolom karena keduanya mewakili siklus yang berbeda: *status* menentukan apakah akun *boleh mulai digunakan* (hasil ACC/tolak admin), sedangkan *account_status* menentukan apakah akun yang sudah aktif *masih diizinkan login* (bisa dinonaktifkan admin sewaktu-waktu tanpa perlu menghapus akun atau mengubah riwayat verifikasinya). *account_status* hanya boleh terisi ketika *status = 'verified'* — ditegakkan lewat *CHECK* constraint dan divalidasi ulang di backend (karena *CHECK* baru berlaku efektif di MySQL 8.0.16+).
- 3. Struktur Lokasi, Fasilitas Tingkat Universitas, dan Fasilitas Outdoor** Tabel *locations* memuat hierarki spesifik (Fakultas, Prodi, Gedung, Ruangan) yang dilengkapi kolom *scope_level* untuk membedakan fasilitas tingkat universitas (mis. lapangan/aula umum kampus yang tidak dinaungi fakultas manapun) dari fasilitas tingkat fakultas. Karena *scope_level* sudah menjadi penanda eksplisit, kolom *fakultas* dibuat nullable tanpa menimbulkan ambiguitas — *NULL* jelas berarti "sengaja tidak ada fakultas" (tingkat universitas), bukan "data belum diisi". Atribut Prodi, Gedung, dan Ruangan juga tetap nullable untuk mengakomodasi fasilitas outdoor (seperti lapangan) yang tidak menempati bangunan fisik.
- 4. Penyimpanan Multi-Foto** Karena laporan kerusakan dan detail fasilitas sering membutuhkan foto dari berbagai sudut, data gambar dipisahkan ke tabel relasional *facility_photos* dan *report_photos* (relasi 1:M).
- 5. Alur "Simpan Draft"** Laporan yang disimpan sementara oleh pengguna akan langsung masuk ke tabel *reports* dengan status *'draft'*. Laporan draf ini tidak akan muncul di dasbor antrean petugas sampai pengguna benar-benar menekan tombol kirim.
- 6. Audit Trail Relasional (*status_logs*)** Rekam jejak perubahan status dipusatkan di tabel *status_logs*. Untuk menjaga integritas database, tabel ini menggunakan tiga Foreign Key terpisah yang nullable (*reservation_id*, *report_id*, *facility_id*), di mana hanya satu yang terisi per baris. Kolom *note* ditambahkan agar riwayat catatan/alasan petugas di setiap tahap tidak tertimpa.
- 7. Presisi Waktu Resolusi** Waktu penutupan laporan direkam permanen di kolom *resolved_at*. Hal ini menjaga akurasi data saat Admin mengeksport rekap kerusakan, karena waktu penyelesaian tidak akan bergeser layaknya *updated_at* jika petugas mengedit typo di kemudian hari.
- 8. Validasi Jadwal Mutlak** Penggunaan tipe data *TIME* pada kolom jam reservasi memastikan validasi jam operasional (07.00–20.00) dan kelipatan slot 30 menit dieksekusi secara ketat oleh server, bukan sekadar batasan di tampilan frontend.
- 9. Keamanan OTP** Sistem pemulihan kata sandi diasumsikan menggunakan Session atau layanan Cache di server, sehingga tabel penyimpanan OTP tidak dimasukkan ke dalam skema database utama.
- 10. Ketersediaan Slot Selama Menunggu Approval** Untuk mencegah race condition (dua pengguna mengajukan reservasi pada slot yang sama sebelum petugas sempat memproses), slot dianggap "occupied" ketika ada reservasi berstatus *'pending'* maupun *'approved'* pada fasilitas, tanggal, dan rentang waktu yang sama. Pengguna lain akan melihat slot tersebut berstatus "menunggu konfirmasi petugas" (bukan tersedia kosong). Jika petugas menolak reservasi tersebut (*status* → *'rejected'*), slot otomatis kembali tersedia untuk pengguna lain.
- 11. Strategi Penghapusan Data — Soft Delete vs Hard Delete vs Status Bisnis** Tidak semua tabel memakai mekanisme yang sama untuk "menghapus" data, disesuaikan dengan seberapa besar dampaknya terhadap integritas riwayat:
 - *users* dan *facilities* memakai **soft delete** (kolom *deleted_at*, lewat trait *SoftDeletes* di Laravel). Kedua entitas ini adalah induk dari banyak data historis (reservasi, laporan, foto, log audit) yang tidak boleh ikut hilang. Dengan soft delete, baris tetap ada secara fisik di database (relasi FK tetap valid), tapi disembunyikan dari query normal

aplikasi — cocok juga dengan kebutuhan Admin mengekspor rekap historis (user story 17) meskipun user/fasilitas terkait sudah "dihapus" dari sisi tampilan.

- **locations dan facility_types** memakai **hard delete biasa**, dilindungi `restrictOnDelete()` pada FK yang merujuknya. Ini sudah cukup aman: MySQL otomatis menolak penghapusan selama masih ada **facilities** yang merujuk ke sana, dan baru bisa dihapus permanen kalau sudah benar-benar tidak dipakai. Tidak perlu kolom tambahan.
- **reservations dan reports** tidak memakai soft delete maupun hard delete untuk "penghapusan" — siklus hidupnya sudah direpresentasikan lewat kolom **status** sendiri ('cancelled'/'rejected' untuk reservasi, 'ditolak' untuk laporan). Menambahkan **deleted_at** di sini akan redundan dan berpotensi ambigu dengan status yang sudah ada.
- **facility_photos dan report_photos** tetap **hard delete dengan cascadeOnDelete()** — foto adalah lampiran, bukan entitas historis mandiri, jadi wajar ikut terhapus saat fasilitas/laporan induknya benar-benar dihapus (atau, untuk **facilities**, tetap ada karena soft delete mempertahankan barisnya).

Catatan Implementasi Tambahan

Dua catatan berikut sifatnya teknis/implementasi (bukan perubahan struktur tabel), jadi tidak dituliskan sebagai asumsi formal di atas, tapi penting untuk diperhatikan saat menulis SQL `CREATE TABLE` maupun kode backend:

- Indexing untuk performa query.** Tambahkan index berikut agar fitur pengecekan bentrok jadwal dan dashboard petugas tetap cepat begitu data reservasi/laporan bertambah banyak:
 - `(facility_id, reservation_date, status)` pada tabel **reservations**
 - `(facility_id, status)` pada tabel **reports**

Tanpa index ini, query overlap-check (approval reservasi) dan query antrian laporan berpotensi melambat signifikan seiring bertambahnya data.
- Validasi "tepat satu FK terisi" di status_logs (opsional, nice-to-have).** Karena **status_logs** memiliki tiga Foreign Key nullable (**reservation_id**, **report_id**, **facility_id**), pastikan di level aplikasi (atau lewat **CHECK** constraint bila menggunakan MySQL 8+/PostgreSQL) bahwa **tepat satu** dari ketiganya terisi per baris. Ini mencegah baris log yang salah kaitan — misalnya ketiganya **NULL**, atau lebih dari satu terisi sekaligus dalam satu baris log yang sama.
- Transaction & re-check saat submit reservasi.** Agar aturan pada Asumsi poin 9 benar-benar aman dari race condition di level database (dua request submit di waktu nyaris bersamaan), proses submit reservasi sebaiknya dibungkus transaction dengan re-check availability tepat sebelum **INSERT**, bukan hanya mengandalkan pengecekan saat halaman di-render.
- Penegakan relasi mandatory-mandatory via transaction (bukan lewat skema/FK).** Untuk dua relasi yang didesain wajib di kedua sisi — **locations**↔**facilities** (setiap lokasi wajib punya ≥1 fasilitas) dan **reports**↔**report_photos** (setiap laporan wajib punya ≥1 foto) — sisi "induk wajib punya anak" **tidak bisa** ditegakkan oleh FK constraint saja, karena baris anak baru bisa dibuat setelah induknya punya **id**. Penegakannya harus di level aplikasi:
 - Submit laporan kerusakan (**reports**) dan upload foto bukti pertamanya (**report_photos**) dibungkus dalam satu **transaction**; jika tidak ada foto yang berhasil di-attach, insert laporan ikut di-rollback (tombol "Kirim" di userflow sebaiknya disabled/divalidasi sampai minimal satu foto terlampir).
 - Penambahan **locations** baru sebaiknya dilakukan bersamaan dengan fasilitas pertamanya dalam satu alur/transaction (atau lokasi disiapkan sebagai data seed awal), dan aplikasi melarang penghapusan fasilitas terakhir dari sebuah lokasi.
- Kompatibilitas CHECK constraint dengan versi database.** Dua **CHECK** constraint yang dipakai di skema ini — **users** (relasi **status** ↔ **account_status**) dan **locations** (relasi **scope_level** ↔ **fakultas**) — baru ditegakkan otomatis oleh MySQL mulai versi **8.0.16 ke atas**; versi lebih lama akan menerima syntax-nya tapi diam-diam mengabaikannya (tidak error, tapi juga tidak memvalidasi). Kalau tim menggunakan MySQL versi lama atau engine lain yang tidak mendukung **CHECK**, validasi dua aturan ini **wajib didobel di sisi backend** sebelum **INSERT/UPDATE**.
- Wiring trait SoftDeletes di Model Eloquent.** Menambahkan kolom **deleted_at** lewat migration saja **tidak cukup** — Model **User** dan **Facility** di Laravel wajib memakai trait `Illuminate\Database\Eloquent\SoftDeletes` (`use`

`SoftDeletes`; di dalam class Model), supaya `->delete()` otomatis menjadi soft delete dan query default (`User::all()`, `Facility::all()`, dst.) otomatis mengecualikan baris yang sudah "terhapus". Tanpa trait ini, kolom `deleted_at` cuma jadi kolom biasa yang tidak berfungsi apa-apa. Kalau sewaktu-waktu Admin butuh melihat/mengembalikan data yang soft-deleted, gunakan method bawaan Laravel `withTrashed()` atau `restore()`.